

TASK 4 – ANALYSE BEST, AVERAGE AND WORST-CASE BEHAVIOUR

1. Introduction

Best-case, average-case and worst-case analysis are used to evaluate the performance of an algorithm under different input conditions. These cases help us understand how the number of operations changes depending on the location of the required registration number and the structure of the data.

In this task, Linear Search, Recursive Binary Search and Hash Table Search are analysed for locating a student registration number in a university database.

Let n represent the total number of student records in the database.

2. Linear Search

Linear Search examines registration numbers sequentially, starting from the first record. Each registration number is compared with the required key until a match is found or the entire database has been checked.

Best Case

The best case occurs when the required registration number is present in the first position.

Only one comparison is required to find the record.

Number of comparisons = 1

Therefore, the best-case time complexity is:

$\Omega(1)$, or $\Theta(1)$ for this input condition.

This occurs because the algorithm finds the required registration number immediately without examining the remaining records.

Average Case

The average case occurs when the required registration number is equally likely to be located at any position in the database, assuming the record exists.

For a successful search, the number of comparisons can range from 1 to n . The average number of comparisons is:

$$\text{Average comparisons} = (1 + 2 + 3 + \dots + n) / n$$

Using the arithmetic-series formula:

$$\text{Average comparisons} = n(n + 1) / 2n$$

$$\text{Average comparisons} = (n + 1) / 2$$

Therefore, the average-case time complexity is $\Theta(n)$.

As the number of records increases, the number of comparisons increases approximately in proportion to the database size.

Worst Case

The worst case occurs when the required registration number is in the last position or is not present in the database.

The algorithm must examine all n records before it can finish.

$$\text{Number of comparisons} = n$$

Therefore, the worst-case time complexity is $\Theta(n)$.

The worst case occurs because Linear Search has no information that allows it to skip records. It must check each record sequentially.

3. Recursive Binary Search

Binary Search repeatedly divides a sorted database into two halves. It compares the required registration number with the middle element and continues searching only in the relevant half.

Precondition: The registration numbers must be sorted before Binary Search is performed.

Best Case

The best case occurs when the required registration number is exactly equal to the middle element during the first comparison.

Number of comparisons = 1

Therefore, the best-case time complexity is $\Theta(1)$.

The algorithm terminates immediately because the required record is found at the midpoint of the search range.

Average Case

The average case occurs when the required registration number is located at different positions within a sorted database, assuming the searched keys and positions are distributed suitably.

After every comparison, approximately half of the remaining records are eliminated. Therefore, the search range decreases as follows:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

After k divisions, the remaining range is approximately:

$$n / 2^k$$

The search terminates when the remaining range contains at most one record:

$$n / 2^k \leq 1$$

Therefore:

$$2^k \geq n$$

$$k \geq \log_2 n$$

Hence, the average-case time complexity is $\Theta(\log n)$ under the usual balanced-search assumptions.

Binary Search is efficient because it does not examine every record. Each comparison reduces the remaining search space by approximately half.

Worst Case

The worst case occurs when the required registration number is not found or is located at a position that requires the maximum number of divisions.

The algorithm continues dividing the search range until the range becomes empty or the record is found.

The number of divisions is proportional to $\log_2 n$.

Therefore, the worst-case time complexity is $\Theta(\log n)$.

Unlike Linear Search, the number of comparisons increases very slowly when the database becomes larger.

Preprocessing consideration: If the records are initially unsorted, sorting must also be considered. An efficient comparison-based sorting algorithm typically requires $\Theta(n \log n)$ time. If the records are already sorted, this preprocessing cost need not be repeated for every search.

4. Hash Table Search

Hash Table Search applies a hash function to a registration number to calculate its corresponding index in a hash table. It then examines that location and uses a